

Session 02 — The ML Framework: Summary

The big idea in one line: machine learning replaces hand-written rules with a *model* whose *weights* are tuned by data to *minimize a loss*, and the real goal is for that model to *generalize* to data it has never seen.

The whole lecture is one recurring recipe — **model** → **loss** → **optimization** → **generalization** — taught through a single running example: a spam classifier.

Topic by topic

1. What machine learning is

Instead of a human writing the rules, we show the machine examples and it learns the rule itself. A program is doing machine learning when it improves at a task as it sees more data, rather than being explicitly programmed. The end goal is always to make good predictions on *new, unseen* data.

2. Three kinds of learning

- **Supervised** — learn from labeled (input, correct-answer) pairs. *Our focus.*
- **Unsupervised** — find structure in data with no labels.
- **Reinforcement** — learn from reward by acting in an environment.

3. Setting up a supervised problem

Every supervised problem has three questions: what is the **input**, what is the **output**, and how do we **measure success**? Two flavors by the kind of answer: **classification** (a category, e.g. email → spam) and **regression** (a number, e.g. house → price). We build the whole lecture on classification.

4. The recipe (the spine)

Every supervised method is the same four choices, all acting on one object — the prediction function f :

1. **Model** — the family of candidate functions f (the *hypothesis class*).
2. **Loss** — how we score how wrong an f is.
3. **Optimization** — how we search the family for the best f .
4. **Generalization** — whether that f works on new data.

5. From email to numbers (feature engineering)

A computer only understands numbers, so we convert each email into **features**. To keep it simple we use two: the number of links and the number of suspicious words. Choosing these numbers is *feature engineering*.

6. The data, and why it is hard

The spam dataset has overlapping classes on purpose — a not-spam email and a spam email can have similar feature counts. Because the classes overlap, **no rule can be perfect**.

7. A first model and where it fails

A simple straight-line rule ("more links and suspicious words → spam") already makes mistakes, because the classes overlap. This motivates two things: a way to *measure* the mistakes, and a way to *reduce* them.

8. The model is a hypothesis class

The linear model $f(x) = w \cdot x + b$ defines a family of straight-line decision boundaries. Learning is a **search**: pick the single boundary in that family that best separates the classes.

9. Measuring error

- **0/1 error** — the fraction of examples the model gets wrong. Simple and honest, but the thing we ultimately care about.
- **Confusion matrix** — splits predictions into true/false positives and negatives; shows *which* mistakes you make, not just how many.
- **Why accuracy lies** — on imbalanced data (e.g. 90% not-spam), "always say not-spam" scores 90% while catching zero spam. A model can be accurate and useless.

10. Generalization

- **True vs empirical risk** — we can only minimize average loss on the training set (empirical), but we are judged on expected loss over all data (true).
- **Train / validation / test** — train to learn f , validation to tune the knobs, test to judge once. Never evaluate on data you trained on.
- **Why generalization is even possible** — train and test come from the same distribution. When that breaks (**distribution shift**), even a good model fails: train on English emails, test on Arabic, and the spam filter won't work.

11. Learning as optimization

"How do we learn f ?" becomes a concrete optimization problem: choose the f that minimizes the average number of mistakes on the data.

12. Logistic regression

A linear score $w \cdot x + b$ is unbounded, but we need a probability. The **logistic (sigmoid) function** $\sigma(z) = 1/(1+e^{-z})$ squashes the score into a probability between 0 and 1: $f(x) = \sigma(w \cdot x + b)$. Predict spam when the probability is above 0.5.

13. The loss from maximum likelihood

The loss is not arbitrary — it is *derived*:

- Model each label as a Bernoulli draw with probability $p_i = \sigma(w \cdot x_i + b)$.
- The **likelihood** of the dataset (how likely the observed labels are, given the model's predicted probabilities) is the product of those terms.
- **Maximum likelihood** chooses the weights that make the data most likely.

- Taking the log turns the product into a sum, and maximizing it equals minimizing its negative average — the **binary cross-entropy loss**.

Cross-entropy is cheap when you are confident and right, and very expensive when you are confident and wrong. It is smooth (differentiable), which the next step needs.

14. Convexity

The cross-entropy loss is **convex**: one bowl, one global minimum, no bad local minima. Maximizing likelihood equals minimizing the negative log-likelihood. At the minimum the gradient is zero ($\nabla L(w^*) = 0$), but there is no closed-form solution — so we descend to find it.

15. The gradient and gradient descent

- **The gradient** points uphill; the key fact is the sigmoid derivative $\sigma'(z) = \sigma(z)(1 - \sigma(z))$, which makes the gradient simplify to a clean form: the average of (prediction – label) times each input.
- **Gradient descent** steps the weights downhill: $w \leftarrow w - \eta \nabla L(w)$, repeated until the loss stops dropping.

16. Learning rate and SGD

The **learning rate** η is the one knob to respect: too small is slow, too big diverges, just right converges. **Batch** uses all data per step (exact, slow); **stochastic (SGD)** uses one example (noisy, fast); **mini-batch** uses a small group (the practical default).

17. Model complexity: features and weights

A straight line can't separate everything. Adding features like x_1^2 , x_2^2 , x_1x_2 lets the boundary curve. Each feature has its own **weight** — so a more flexible (easily overfit) model is one with more features and more weights.

18. Over/underfitting and bias-variance

- **Underfit** — too simple, misses the pattern (high bias).
- **Overfit** — too complex, wraps around noise (high variance).
- Detect it with the **train/test gap**: the sweet spot is the bottom of the U-shaped test-error curve, not the bottom of the training curve.
- **Bias-variance tradeoff**: expected error = bias² + variance + irreducible noise.

19. Regularization (L1 / L2)

Add a penalty to the loss so those extra weights can't grow large and bend the boundary too much. **L2 (ridge)** $\lambda \|w\|^2$ shrinks weights smoothly; **L1 (lasso)** $\lambda \|w\|_1$ drives some to exactly zero (feature selection). λ controls the strength and is tuned on validation.

Appendix — data leakage

Information that won't exist at prediction time sneaking into training. It produces "too-good-to-be-true" scores. The fix: split first, then touch the data.

Recap: the four boxes

Box	In this lecture
Model	Logistic regression: $f(x) = \sigma(w \cdot x + b)$, a hypothesis class of (possibly curved) boundaries.
Loss	Binary cross-entropy, derived from maximum likelihood.
Optimization	Gradient descent on a convex loss, tuned by the learning rate.
Generalization	Train/val/test, the bias-variance tradeoff, and regularization.

Said as one sentence: *choose a class of $f \rightarrow$ score it with a loss $\rightarrow$ search for the best f with gradient descent $\rightarrow$ check that it generalizes.*

Lessons learned

1. **Learning is rules from data, not data from rules.** The whole field is one inversion of traditional programming.
2. **It's always the same four boxes.** Model, loss, optimization, generalization — every method, including the large models, is an instance of this recipe.
3. **Losses aren't guessed, they're derived.** Cross-entropy is the negative log-likelihood of a Bernoulli model; the math tells you what to minimize.
4. **We can't optimize what we care about directly.** The 0/1 error is flat, so we relax it into a smooth, convex surrogate and descend.
5. **Fitting the data is not the goal — generalizing is.** Training accuracy can lie (overfitting); honest evaluation and regularization are what make a model trustworthy.
6. **Complexity lives in the weights.** Curved boundaries come from extra features, each with a weight; regularization is just keeping those weights in check.
7. **The assumptions matter.** Generalization rests on train and test sharing a distribution — break that (distribution shift) and even a good model fails.

Next: representations (Session 3), then deep learning (Session 4) — the same recipe with a richer model class.